Flood Prediction System Using Machine Learning

Project Description:

The Flood Prediction System Using Machine Learning is an intelligent web application that predicts the likelihood of flood occurrence based on environmental and weather-related parameters. The system enables users to input factors such as rainfall, temperature, humidity, river water level, and seasonal conditions to obtain accurate flood risk predictions.

The application utilizes machine learning algorithms such as Random Forest, Decision Tree, K-Nearest Neighbors (KNN), and XGBoost to analyze historical flood datasets and generate reliable predictions. Built using the Flask framework, the application provides a simple and interactive web interface for users to access prediction services in real time.

The system performs data preprocessing, feature analysis, model prediction, and result visualization while ensuring efficient processing and user-friendly interaction. It can assist disaster management authorities, researchers, and local communities in taking preventive measures against potential flood disasters.

Scenarios

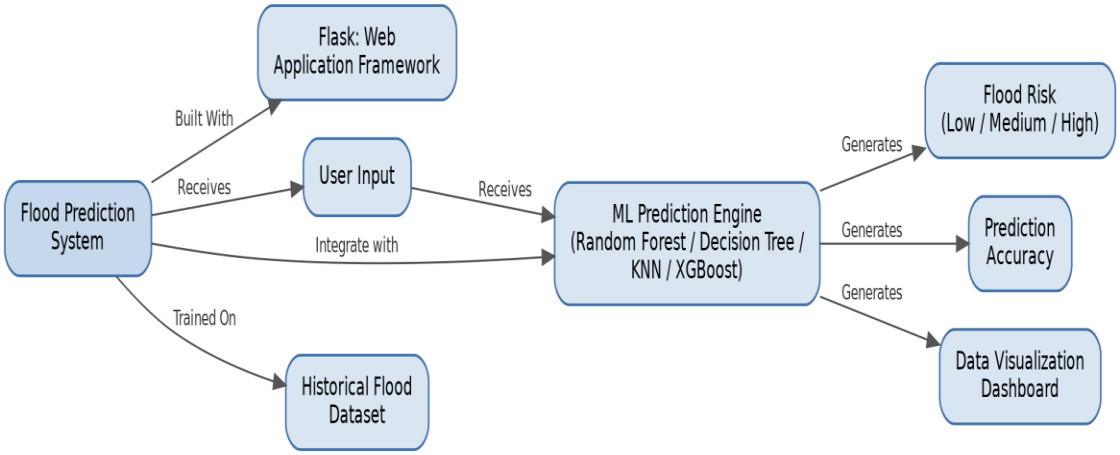
Scenario 1: A disaster management officer enters rainfall, humidity, river water level, and seasonal information collected from a flood-prone region. The system processes the input using the trained machine learning model and predicts a High Flood Risk, allowing authorities to issue early warning alerts.

Scenario 2: A researcher analyzes historical environmental data from different regions. The system predicts flood probability for each dataset and displays the results, helping researchers identify high-risk areas for future planning.

Scenario 3: A local government official monitors weather conditions during the monsoon season. By entering updated environmental parameters into the application, the system predicts the flood risk level, enabling timely evacuation planning and disaster preparedness.

Technical Architecture:

Description: The Flood Prediction System follows a modular three-tier architecture for efficient data processing and prediction. The Presentation Layer consists of a responsive web interface developed using HTML, CSS, and JavaScript, allowing users to enter environmental parameters and view prediction results. The Application Layer is implemented using the Flask framework, which handles user requests, validates input data, performs preprocessing, loads the trained machine learning model, and generates flood predictions. The Data Layer contains the historical flood dataset and the trained machine learning model, and applies algorithms such as Random Forest, Decision Tree, KNN, and XGBoost to analyze input data and predict flood occurrence.



Note: If a database is used in your implementation, include the ER Diagram along with its description.

Pre-requisites:

- **Python Programming Knowledge:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Basics:** Scikit-learn Documentation
- **Pandas & NumPy:** Data Processing Libraries
- **HTML, CSS & JavaScript Skills:** W3Schools Tutorials
- **Version Control using Git:** Git Documentation
- **Development Environment:** Visual Studio Code
- **Model Training Libraries:** Scikit-learn, XGBoost
- **Python Package Management:** pip and requirements.txt

Project Workflow:

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate machine learning algorithms for flood risk prediction. This involves researching the capabilities of Random Forest, Decision Tree, KNN, and XGBoost, ensuring that the chosen models align well with the application's objective of classifying flood risk as Low, Medium, or High.

- **Activity 1.1:** Collect and explore the historical flood dataset containing rainfall, temperature, humidity, river water level, and seasonal records.
- **Activity 1.2:** Research and shortlist candidate ML algorithms (Random Forest, Decision Tree, KNN, XGBoost) suited for a classification task of this kind.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, Flask backend, and the trained ML model.

- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, scikit-learn, XGBoost, pandas, and NumPy.

Milestone 2: Data Preprocessing and Model Training

Milestone 2 covers preparing the historical flood dataset for training and building the prediction models that power the application.

Activity 2.1: Data Cleaning and Feature Engineering

- Handle missing values and outliers in rainfall, humidity, temperature, and river water level readings.
- Encode the seasonal feature (e.g., monsoon, winter, summer) into a numerical representation.
- Scale/normalize continuous features so that all models train on comparable ranges.

Activity 2.2: Train and Evaluate Candidate Models

- Split the dataset into training and testing sets.
- Train Random Forest, Decision Tree, KNN, and XGBoost classifiers on the processed data.
- Evaluate each model using accuracy, precision, recall, F1-score, and a confusion matrix.

Activity 2.3: Select and Persist the Final Model

- Compare model performance and select the best-performing algorithm for production use.
- Serialize the trained model and any preprocessing objects (scaler/encoder) using joblib or pickle for later loading in the Flask app.

Milestone 3: App.py Development

Milestone 3 focuses on creating the core logic of the app.py file, which serves as the backbone of the Flood Prediction System. This includes defining Flask routes, loading the trained model, and building the prediction pipeline.

Activity 3.1: Define the Core Routes in app.py

- / — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- /predict — POST endpoint that accepts form/JSON input, runs the ML model, and returns the predicted flood risk

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    rainfall = float(data.get("rainfall", 0))
    temperature = float(data.get("temperature", 0))
    humidity = float(data.get("humidity", 0))
    river_level = float(data.get("river_level", 0))
    season = data.get("season", "monsoon")
```

```
    try:
        features = preprocess_input(rainfall, temperature, humidity, river_level, season)
        prediction = model.predict(features)[0]
        risk_label = RISK_LABELS.get(prediction, "Unknown")
        return jsonify({
            "success": True,
            "flood_risk": risk_label
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Activity 3.2: Load the Trained Model at Startup

- Load the serialized model and preprocessing objects once when the Flask app starts, so predictions stay fast on every request.

```
import joblib
model = joblib.load("model/flood_model.pkl")
scaler = joblib.load("model/scaler.pkl")
```

Activity 3.3: Input Validation and Error Handling

- Validate that all required fields (rainfall, temperature, humidity, river_level, season) are present and numeric where applicable.
- Return a 400 error with a clear message when validation fails, and a 500 error if prediction fails unexpectedly.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, responsive interface for the Flood Prediction System, and wiring it to the backend using JavaScript.

Activity 4.1: Designing and Developing the User Interface

- Develop the main HTML file (index.html) as a single-page interface, with an input form for rainfall, temperature, humidity, river water level, and season.
- Create a CSS stylesheet implementing a clean, weather-themed layout with clear visual indicators for Low/Medium/High risk.
- Add media queries so the layout adapts across desktop, tablet, and mobile screen sizes.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

- Attach a submit event listener to the prediction form that sends the input as a JSON POST request to /predict via the Fetch API.
- While the request is processing, disable the submit button and show a loading indicator.
- Render the returned flood risk level dynamically, using color-coded badges (green/yellow/red) for Low/Medium/High risk, along with a simple results dashboard/chart.
- In the finally block of the async handler, re-enable the submit button regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Flood Prediction System first locally to verify correct operation, and then to a public host so it can be shared and used by disaster management teams.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Activity 5.2: Local Testing and Verification

- Run the application locally and open a browser to access the app.

```
python app.py
```

- Test the prediction form with a range of rainfall, humidity, river level, and seasonal combinations to verify the model responds correctly across Low, Medium, and High risk cases.

Activity 5.3: Public Deployment

- Deploy the application to a hosting platform (e.g., Render, PythonAnywhere, or a cloud VM) so disaster management authorities and researchers can access it from anywhere.
- Configure environment variables and a production-ready WSGI server (e.g., Gunicorn) instead of the Flask development server.

Milestone 6: Conclusion

Milestone 6 wraps up the project by reviewing the complete pipeline — from data preprocessing and model training through to a deployed, user-facing prediction tool — and identifying opportunities for future enhancement.

Exploring the Website's Web Pages:

Home / Prediction Page:

Description: The single-page interface of the Flood Prediction System serves as both the landing page and the prediction tool. It features a clean, weather-themed design with a header displaying the application name and a short tagline. The main section contains the input form and the prediction results panel, all on one seamless page.

Input Section:

Description: A card containing the main prediction form. Users enter rainfall, temperature, humidity, and river water level as numeric values, and select the current season from a dropdown or toggle. Clicking the Predict Flood Risk button sends the data to the backend; while the model processes the request, the button shows a loading state.

Results Section:

Description: Revealed dynamically after prediction, this section displays the flood risk level (Low, Medium, or High) as a color-coded badge, along with a short explanation and a simple visualization of the input parameters relative to typical safe ranges. The viewport automatically scrolls to this section once a prediction is generated.

Conclusion:

The Flood Prediction System Using Machine Learning is a practical, Flask-based platform that turns environmental readings — rainfall, temperature, humidity, river water level, and season — into an easy-to-understand flood risk assessment. By training and comparing Random Forest, Decision Tree, KNN, and XGBoost models on historical flood data, the system delivers reliable Low/Medium/High risk predictions in real time. The project, which spanned model selection, data preprocessing, backend and frontend development, and deployment, shows how a focused machine learning pipeline can support disaster management authorities, researchers, and local communities in taking timely preventive action. Looking ahead, the platform has room to grow — integrating live weather-API feeds, adding satellite or river-gauge imagery for richer inputs, and introducing user accounts to track predictions over time would help it evolve from a prediction tool into a comprehensive early-warning system.